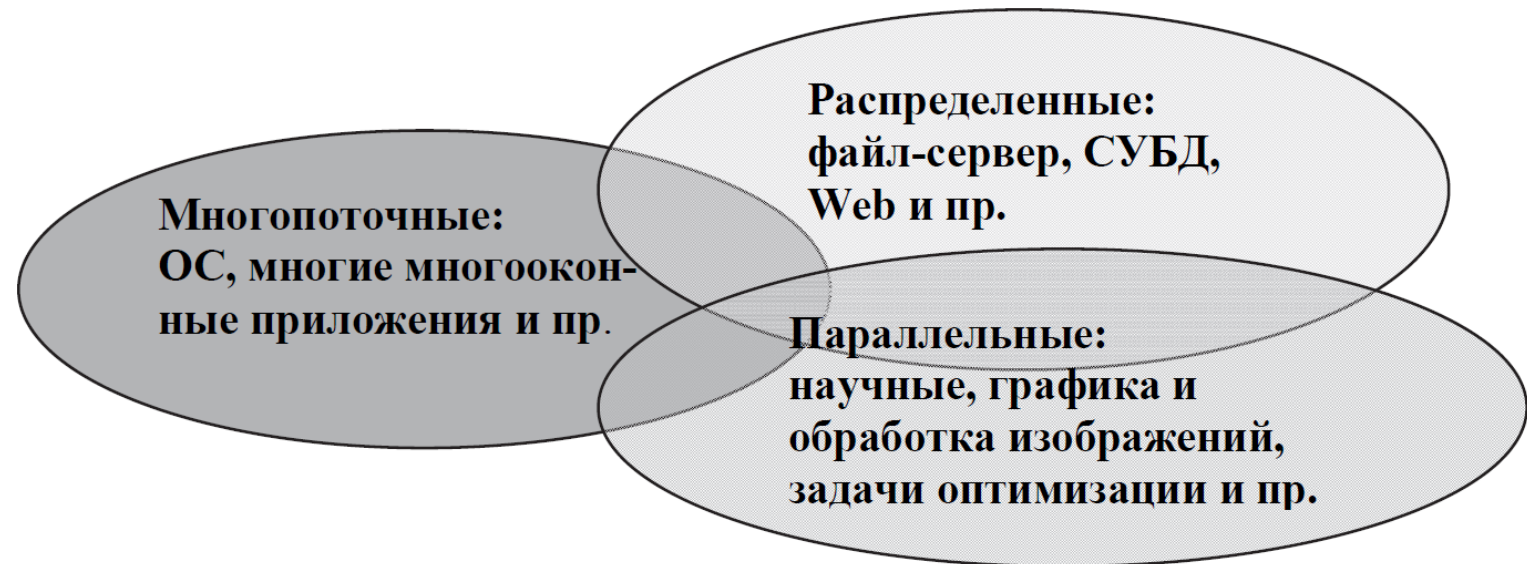


Разработка параллельных
приложений.

Разработка параллельных приложений

- Существуют три широко используемых и частично перекрывающихся подхода к организации параллельных приложений, определяющих три соответствующих им класса программ



1. Многопоточные приложения.

- Термин многопоточный обычно означает, что программа содержит больше процессов/потоков, чем существует процессоров для их выполнения. В результате некоторые процессы приложения выполняются по очереди.
- Примерами многопоточных программ являются операционные системы (ОС), системы разделения времени, системы реального времени, многие оконные приложения. Эти системы разрабатываются многопоточными, поскольку составляющие их задачи могут планироваться и выполняться независимо.
- **Основной проблемой реализации таких программ является борьба за разделяемые ресурсы, например, процессор, память, устройства ввода-вывода и другие.**

2. Распределенные системы.

- В таких системах компоненты программ выполняются на разных вычислительных машинах, связанных сетью, возможно, работающих под разными ОС. Основным способом взаимодействия процессов в распределенных системах является обмен сообщениями. Спецификой процессов является их относительная независимость и возможность обслуживания одним процессом различных клиентов. Подобное обслуживание обычно происходит по поступающим запросам.
- Примерами распределенных систем могут служить файловые серверы в сети, СУБД, веб-серверы в сети Интернет, отказоустойчивые системы, которые продолжают работать независимо от сбоев в компонентах.
- Распределенные системы необходимы для организации доступа к территориально удаленным данным, интеграции и управления данными, распределенными по своей сути, повышения надежности. Как правило, такие системы основаны на технологии клиент-сервер, а их компоненты сами являются многопоточными программами.
- **Основной проблемой реализации распределенных программ является обмен информацией о состоянии разделяемого ресурса.**

3. Параллельные (согласованные) вычисления.

- Цель таких приложений – совместно и быстро решить данную задачу, причем допустимы как многопоточный, так и распределенный подходы.
- Примерами согласованных параллельных вычислений являются различные научные вычисления (например, численные модели математической физики), графика и обработка изображений (включая создание спецэффектов в кино), крупные комбинаторные и оптимизационные задачи, интеллектуальный анализ данных.
- Параллельные программы требуют больших вычислительных мощностей. Обычно число процессов в параллельных программах равно числу процессоров (ядер).

Параллелизм по данным

- В таксономии Флинна уже прослеживается два основных подхода к построению параллельных приложений.
- 1. Параллелизм по данным основан на выполнении процессами (потоками) одних и тех же действий над собственной частью данных.
- В этом случае программист задает опции векторной или параллельной оптимизации, директивы параллельной компиляции, а собственно векторизацию или распараллеливание выполняет транслятор. Существуют также специализированные языки параллельных вычислений. Параллелизм по данным реализует SIMD-технологии. Для такого подхода характерно, что обработкой данных управляет одна программа, но процессы слабо синхронизированы (каждый процесс выполняет один код, но нет гарантии, что в заданный момент времени все процессы выполняют одну и ту же инструкцию).

2. Параллелизм по задачам

- предусматривает разбиение вычислительной задачи на несколько самостоятельных подзадач. Компьютер при этом представляет MIMD-машину. Для каждой подзадачи, как правило, пишется своя программа. Подзадачи в ходе работы обмениваются данными, а также согласуют свою работу. При программировании контролируется распределение данных между процессами/потоками и вычислительной нагрузки между процессорами, что повышает трудоемкость разработки и отладки программы.
- Как правило, выбор метода распараллеливания основан на свойствах самой задачи или выбранного алгоритма ее решения. Так, для сеточных вычислений характерен параллелизм по данным, однако можно выбрать такой алгоритм решения (например, метод прогонки для решения СЛАУ с трехдиагональной матрицей), который испортит весь параллелизм исходной задачи. Задача обработки изображения может обладать внутренним параллелизмом по задачам, а каждая задача – параллелизмом по данным.

Декомпозиция – связь – синхронизация (ДСС)

- Процесс проектирования параллельных приложений обязательно включает следующие три составляющие: декомпозиция – связь – синхронизация (ДСС).
- Декомпозиция – это процесс разбиения прикладной задачи на части.
- Иногда декомпозиция естественным образом следует из природы самой задачи, иногда она определяется разработчиком.
- Можно проводить разбиение
 - на основе логики действий (например, сортировка, поиск, ввод-вывод, вычисление),
 - на основе логики ресурсов (например, работа с файлом, принтером, базой данных),
 - на основе логики данных (например, обработка массива по строкам, столбцам или блокам).

- Чем меньше размер подзадач, полученных на этом этапе, и чем больше их количество, тем более гибким получается параллельный алгоритм и тем легче обеспечить равномерную загрузку процессоров. При необходимости всегда можно провести обратную операцию – укрупнение.

- Параллелизм можно организовать на уровне команд. Этот вид параллелизма обычно обеспечивается компилятором, ОС и внутренней архитектурой процессора.
- Следующий уровень параллелизма в итеративных программах – цикл. Не всякий цикл можно выполнять параллельно. Решение об этом принимает, как правило, программист, например, с помощью директив компиляции.
- Дальнейшее использование параллелизма проводится на уровне подпрограмм (функций). Решение подзадач можно оформить отдельными функциями, выполнение которых распределяется по потокам. При наличии нескольких процессоров выполнение ряда потоков может протекать параллельно. Этот уровень требует тщательного проектирования синхронизации и взаимодействия, которые, в свою очередь, во многом определяются архитектурой вычислительной системы.

- Как параллелизм этого же уровня можно представить и некоторую разновидность асинхронного выполнения SIMD-программ, при котором один и тот же код программы запускается на разных вычислительных узлах среды с распределенной памятью, поддерживающей обмен сообщениями.
- Дальнейшее укрупнение уровня параллелизма – объекты и приложения – характерно для распределенных приложений.

Связь

- Планирование связей, или коммуникаций, между частями приложения – следующий этап проектирования. Методы связывания, как правило, определяются архитектурой вычислительной системы (разделяемые переменные, каналы, передача сообщений и т. д.), а их конкретная программная реализация диктуется выбранным инструментом программирования – языком или библиотекой.

Синхронизация

- Поскольку множество потоков программы работают в рамках одной задачи, то их функционирование необходимо координировать, поэтому важным моментом проектирования является синхронизация.
- Следует предусмотреть, какие задачи могут выполняться параллельно, а какие – последовательно и в каком порядке; если некоторые части программы закончили работу раньше, то следует ли им заняться выполнением других заданий. Необходимо учесть и то, как разные подзадачи «узнают», что решение достигнуто.

«Гонка данных»

- При некорректной организации связей и синхронизации обычно появляются следующие проблемы.
- «Гонка данных» возникает из-за того, что несколько процессов одновременно пытаются обновить и использовать общие данные. Проблема наиболее очевидна, если данные одновременно перезаписываются. Однако «гонка данных» часто возникает и в случае, когда один процесс неоднократно использует данные, обновляемые другим процессом. При отсутствии согласованности задача-потребитель может использовать устаревшие данные (при опережающем чтении) или потерять данные (производитель может при записи затереть еще не использованные данные).

«Ситуация бесконечной отсрочки»

- Некорректное планирование последовательности выполнения задач приводит и к другой проблеме – «живой блокировке», при которой задача может никогда не получить возможность выполниться. Также эту проблему иногда называют «ситуацией бесконечной отсрочки».

«Мертвая (взаимная) блокировка»

- еще одна ловушка, связанная с ожиданием. В простейшем случае взаимная блокировка возникает, если «задача 1» ждет данных (сигнала) от «задачи 2», которая, в свою очередь, ожидает данных (сигнала) от «задачи 1». В общем случае ситуация может оказаться еще более запутанной.
- Все эти проблемы должны быть выявлены и разрешены на стадии проектирования параллельного приложения.

Парадигмы параллельных приложений

- **Итеративный параллелизм** используется для реализации параллелизма в итеративной программе (чаще всего в циклах). Такой параллелизм характерен для распараллеливания по данным в согласованных параллельных вычислениях.

Рекурсивный параллелизм

- используется в программах с одной или несколькими рекурсивными процедурами, вызов которых независим. Каждый рекурсивный вызов порождает один или несколько новых процессов, которые независимо работают над решением задачи. В рамках такой парадигмы часто реализуются технологии «разделяй и властвуй» или «перебор с возвратом».
- Итеративный и рекурсивный параллелизм основан на приемах, известных в последовательном программировании.

«Производители и потребители»

- модель взаимодействия неравноправных процессов по поводу общих данных. Одни процессы «производят» данные, другие – их «потребляют». Часто такие процессы организуются в конвейер, через который проходит информация. Каждый процесс конвейера потребляет выход своего предшественника и производит входные данные для своего последователя. Другой распространенный способ организации потоков – древовидная структура, на ней основан, в частности, принцип дихотомии.

«Клиенты и серверы»

- наиболее распространенная модель взаимодействия процессов в распределенных системах. Клиентский процесс запрашивает (возможно, неоднократно) данные у сервера и ожидает ответа, затем использует полученные данные по своему усмотрению. Серверный процесс ожидает запроса от клиента, далее в соответствии с поступившим запросом обрабатывает данные и возвращает запросившему их процессу-клиенту. Таким образом, в отличие от предыдущего случая, между клиентом и сервером необходимо установить двустороннюю связь. Сервер может быть реализован как одиночный процесс, обслуживающий одновременно несколько клиентских процессов. Сервер может быть многопоточной программой, каждый поток которой обслуживает своего клиента. Если клиент и сервер выполняются на одном компьютере, то они представляют собой параллельное программное обобщение процедур: сервер исполняет роль процедуры, а клиент ее вызывает. Однако если коды клиента и сервера разнесены в пространстве, то для синхронизации используются специальные технологии, такие как удаленный вызов процедур или рандеву.

«Управляющий и рабочие»

- модель организации вычислений, при которой существует поток, координирующий работу всех остальных потоков. Как правило, управляющий поток распределяет данные, собирает и анализирует результаты. Эта парадигма часто применяется в задачах оптимизации и статистической обработки информации, при обработке изображений и других научных вычислениях с итеративными алгоритмами.

«Взаимодействующие равные»

- модель, в которой исключен не занимающийся непосредственными вычислениями управляющий поток. Распределение работ в таком приложении либо фиксировано заранее, либо динамически определяется во время выполнения.

Портфель задач

- Одним из распространенных способов динамического распределения работ при создании программ для ВС с общей памятью является портфель задач. Портфель задач, как правило, реализуется с помощью разделяемой переменной, доступ к которой в один момент времени имеет только один процесс. Если же память ВС распределенная, то такая схема распределения работ превращается в схему «управляющий – рабочий», поскольку портфель задач оформляется отдельным процессом. Основными примерами в этой области являются научные вычисления с итеративными алгоритмами и системы, требующие децентрализованного принятия решений.

Нотация для определения параллельных вычислений

- Множество чтения операции – это множество переменных, которые в ходе операции читаются, но не изменяются.
- Множество записи операции – это множество переменных, которые в ходе операции изменяются (и, возможно, читаются).
- Две операции называются независимыми, если их множества записи не пересекаются.
- Две операции могут выполняться параллельно, если они независимы, т. е. процессы всегда могут безопасно читать переменные, которые не изменяются. Однако двум процессам небезопасно выполнять запись в общую переменную или одному процессу читать переменную, которую изменяет другой процесс.
- Для определения параллельного выполнения нескольких различных потоков команд примем следующую нотацию (оператор `par` – от «parallel»):
 - `par`
 - `{ ... ;}` // группа операторов 1
 - `\ \` // разделитель различных ветвей оператора
 - `{ ... ;}` // группа операторов 2
 - `endpar`

Для независимого выполнения тела цикла для каждого значения счетчика оператор `par` будем записывать в следующем виде:

```
par for (i=0; i<n; i++) {  
... // параллельное выполнение тела цикла для  
каждого i      }
```

Параллельно потоки команд или тело цикла могут выполняться как реально, так и только теоретически, что зависит от числа процессоров при реализации.

Другой способ определить параллельные вычисления – использовать декларацию `thread` (оператор `par`, выполняемый как фоновый):

```
thread t_name{  
... // код процесса, который может выполняться  
// параллельно с другими процессами }
```

Можно также определить группу процессов, выполняющую совместно (но не синхронно!) один и тот же код:

```
thread t_name (i=0; i<P) {  
... // параллельное выполнение каждого из n процессов  
}
```

Когда встречается данная команда, создается новый процесс (группа процессов), который начинает выполняться, возможно, параллельно.

Предполагается, что при создании группы процессов существует механизм определения процессом размера группы P и своего номера i в группе, процессы в группе нумеруются подряд.

Еще раз подчеркнем, что в рамках данной нотации не обсуждается различие между потоком и процессом. Далее просто будем считать, что при моделировании систем с общей памятью процессы могут разделять переменные, т. е. возможно описание глобальных переменных, доступных всем процессам, а при моделировании с распределенной памятью процессы разделяют только каналы, обмен данными происходит с помощью примитивов обмена сообщениями `send` и `receive`.

Если за декларацией процесса `thread t_name (...) {...}` существуют операторы, то они выполняются параллельно с новыми процессами, операторы, следующие за телом декларации `par`, выполняются только после завершения всех процессов, порожденных в `par`, т. е. эта нотация предполагает в конце неявную синхронизацию барьером. Основное отличие двух объявлений параллельного выполнения в том, что реализация декларации `thread t_name {...}` требует средств явного порождения процесса, а нотация `par` – наличия, например, опций компилятора, указывающего, что данный цикл должен выполняться параллельно. Последний подход реализован, в частности, в технологии OpenMP.

Итеративный параллелизм: произведение матриц

Одной из классических задач, используемых обычно для демонстрации параллелизма по данным в согласованных параллельных вычислениях, является перемножение матриц.

Задача: Даны матрицы A и B размерностью $n \times n$. Вычислить произведение матриц, поместив результат в матрицу C размерностью $n \times n$. Пусть n объявлена и инициализирована, пусть также исходные матрицы A , B и матрица результатов C являются разделяемыми, матрицы A и B уже инициализированы:

```
int n=10000 ; //объявление и инициализация n
double a[n][n], b[n][n], c[n][n];
... //инициализация матриц A и B
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        c[i][j] = 0.0;
        for (k=0; k<n; k++)
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
    }
}
```

- Поскольку множества записи внутренних произведений не пересекаются, их можно выполнять параллельно. Такие алгоритмы обычно называют алгоритмами с массовым параллелизмом.
- Возможны варианты, когда параллельно вычисляются результирующие строки, столбцы или группы строк и столбцов. Например, если выполнять тело цикла по i для каждого значения счетчика параллельно, то параллельно будут насчитываться строки произведения. Поменяв циклы по i и по j местами (это возможно из-за независимости операций тела вложенных циклов по i и по j) и выполнив параллельно тело цикла для каждого значения счетчика j , получим вариант программы, насчитывающий параллельно столбцы. Более того, можно параллельно насчитывать каждый элемент результирующей матрицы.

Пример 1.1. Вычисление произведения матриц по строкам

```
int n= ; //объявление и инициализация n
double a[n][n], b[n][n], c[n][n];
... //инициализация массивов a и b
par for (i=0; i<n; i++){ // параллельное вычисление строк
    for (j=0; j<n; j++){
        c[i][j] = 0.0;
        for (k=0; k<n; k++)
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
    }
}
```

Пример 1.1 демонстрирует параллельное вычисление произведения матриц по строкам. Выполнение оператора `par` здесь порождает столько потоков, сколько строк в матрице A. Распределение потоков по процессорам зависит от среды выполнения и реализации.

- Предположим, что процессы в системе могут разделять переменные. Разделим матрицу на полосы строк, для каждой полосы создадим свой рабочий процесс. Пусть число запускаемых для решения задачи процессов известно системе и равно P ($P < n$, n нацело делится на P), все процессы выполняют один код (пример 1.2). Поскольку множества записи процессов не пересекаются, то процессы корректно вычислят произведение матриц.

Пример 1.2. Вычисление произведения матриц по полосам

```
int n= ; // объявление и инициализация n
double a[n][n], b[n][n], c[n][n];
... // инициализация массивов a и b
thread worker (w=0; w<P){//полосы вычисляются параллельно
//Номер первой строки полосы для процесса w
    int first = w*n/P;
    //Номер последней строки в полосе для процесса w
    int last = first + n/P - 1;
    for (i=first; i<=last; i++)
        for (j=0; j<n; j++){
            c[i][j] = 0.0;
            for (k=0; k<n; k++)
                c[i][j] = c[i][j] + a[i][k] * b[k][j];
        }
}
```

Управляющий и рабочие: распределенное умножение матриц

При обсуждении итеративного параллелизма рассматривался алгоритм параллельного умножения матриц с помощью процессов, которые разделяют общие переменные.

На машинах с распределенной памятью любая переменная должна быть локальной для некоторого процесса и может быть доступна только этому процессу. Для обмена информацией процессы используют передачу сообщений.

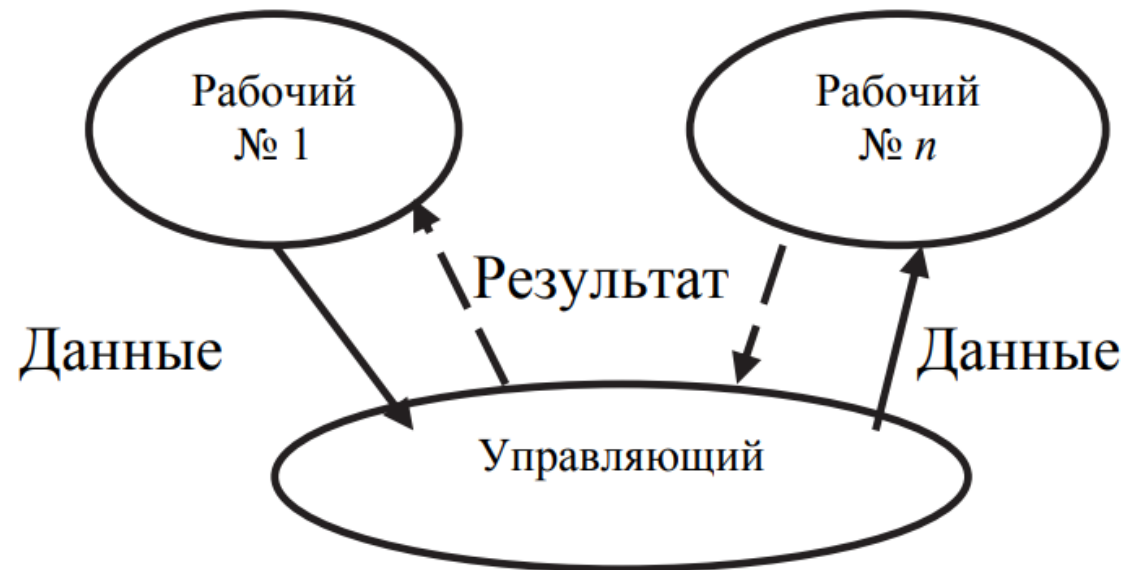
Рассмотрим два способа реализации алгоритма умножения матриц для архитектур с распределенной памятью. В первом случае процессы будут неравноправны – выделен управляющий процесс и пул независимых процессов-вычислителей.

Во втором случае имеет место взаимодействие процессов-вычислителей между собой, которое обеспечивается круговым конвейером. Кроме того, каждый вычислитель взаимодействует с управляющим процессом.

Пусть n объявлено и инициализировано. Пусть также имеется n рабочих процессов, каждый из которых вычисляет одну строку результирующей матрицы.

Для этого каждый рабочий процесс должен располагать одной строкой матрицы A и всей матрицей B . Пусть также существует управляющий процесс, который посылает каждому рабочему процессу необходимые данные и собирает результаты расчетов, т. е. строки матрицы C .

- Операторы `send` и `receive` – это нотация примитивов передачи сообщений. Операция `send` упаковывает сообщение и пересылает его другому процессу; операция `receive` ожидает сообщение от другого процесса, получает его, распаковывает и сохраняет в локальных переменных.



Пример 1.3. Умножение матриц для ВС с распределенной памятью. Взаимодействие «управляющий – рабочий». Код рабочего процесса

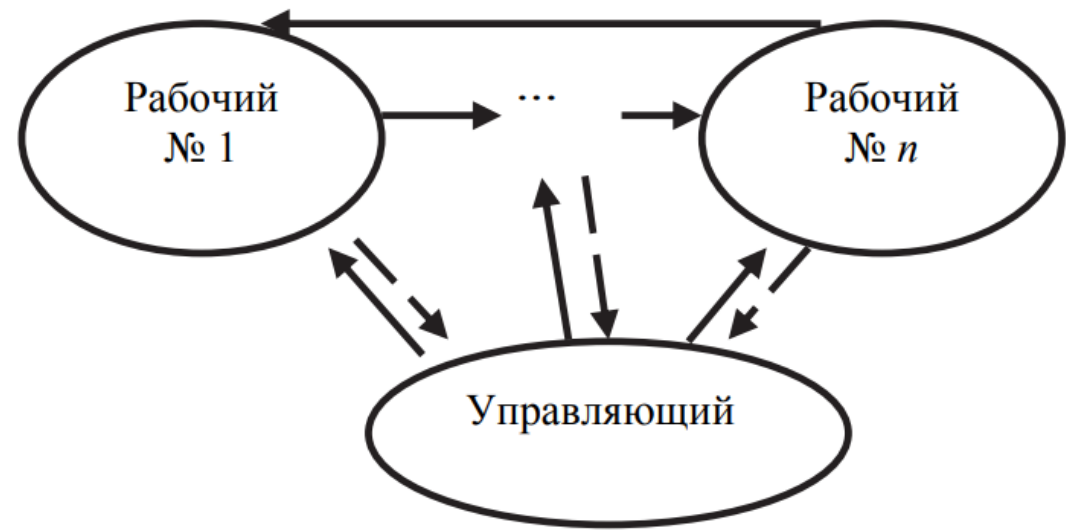
```
thread worker (i=0; i<n) {  
    double a[n];      // строка i матрицы A  
    double b[n][n];   // вся матрица B  
    double c[n];      // строка i матрицы C  
    receive значения элементов массива a, соответствующих i-й  
    строке матрицы A;  
    receive значения массива b;  
    for (j=0; j<n; j++) {  
        c[j] = 0.0;  
        for (k=0; k<n; k++)  
            c[j] = c[j] + a[k] * b[k][j];  
    }  
    send вектор-результат к управляющему процессу;  
}
```

Пример 1.4. Умножение матриц для ВС с распределенной памятью. Взаимодействие «управляющий – рабочий». Код управляющего процесса

```
thread manager {  
    double a[n][n]; // исходная матрица A  
    double b[n][n]; // исходная матрица B  
    double c[n][n]; // результирующая матрица C  
    инициализировать a и b;  
    for (i=0; i<n; i++) {  
        send строку i массива a процессу worker[i];  
        send весь массив b процессу worker[i];  
    }  
    for (i=0; i<n; i++)  
        receive строку i массива c от процесса worker[i];  
    вывести результат, который теперь в массиве c;  
}
```

Круговой конвейер

- Пусть теперь рабочие процессы изначально имеют по i -й строке матрицы A и по j -му столбцу матрицы B .
- Все рабочие процессы соединены в круговой конвейер, по которому циркулируют столбцы матрицы B .



Пример 1.5. Умножение матриц для ВС с распределенной памятью. Взаимодействие управляющий-рабочий, круговой конвейер. Код рабочего процесса

```
thread worker (w=0; w<n) {
    double a[n]; // одна строка матрицы A
    double b[n]; // один столбец матрицы B
    double c[n]; // одна строка матрицы C
    double sum = 0.0; // для промежуточных произведений
    int next_col = w; // следующий столбец результатов,
                      // сначала равен номеру процесса w

    receive строку w массива a и столбец w массива b
        от управляющего процесса;

    // вычислить  $c[w,w] = a[w,*] * b[:,w]$ 
    for (k=0; k<n; k++)
        sum = sum + a[k] * b[k];
    c[next_col] = sum;

    // пустить по кругу столбцы и вычислить остальные  $c[j,*]$ 
    for (j=1; j<n; j++) {
        send мой столбец матрицы b следующему процессу;
        receive новый столбец матрицы b от предыдущего процесса;
        sum = 0.0;
        for (k=0; k<n; k++)
            sum = sum + a[k] * b[k];
        if (next_col == 0) next_col = n - 1;
        else next_col = next_col - 1;
        c[next_col] = sum;
    }
    send вектор-результат управляющему процессу;
}
```

1						
	1					
		1				
			1			
				1		
					1	
						1

1						2
2	1					
	2	1				
		2	1			
			2	1		
				2	1	
					2	1

1					3	2
2	1					3
3	2	1				
	3	2	1			
		3	2	1		
			3	2	1	
				3	2	1

1				4	3	2
2	1				4	3
3	2	1				4
4	3	2	1			
	4	3	2	1		
		4	3	2	1	
			4	3	2	1

1			5	4	3	2
2	1			5	4	3
3	2	1			5	4
4	3	2	1			5
5	4	3	2	1		
	5	4	3	2	1	
		5	4	3	2	1

1		6	5	4	3	2
2	1		6	5	4	3
3	2	1		6	5	4
4	3	2	1		6	5
5	4	3	2	1		6
6	5	4	3	2	1	
	6	5	4	3	2	1

1	7	6	5	4	3	2
2	1	7	6	5	4	3
3	2	1	7	6	5	4
4	3	2	1	7	6	5
5	4	3	2	1	7	6
6	5	4	3	2	1	7
7	6	5	4	3	2	1

- Для запуска кругового конвейера требуется процесс-инициатор вычислений. Его код будет отличен от всех остальных. Этот процесс может быть управляющим и заниматься только первоначальной рассылкой и окончательной сборкой результатов, тогда его код будет такой же, как и в предыдущем случае.
- В чем же различие двух алгоритмов? В первом случае значения матрицы B дублируются в памяти каждого рабочего процесса. Во втором алгоритме в каждый момент времени каждый процесс хранит только одну строку матрицы A и один столбец матрицы B . Это снижает затраты на память процесса, но вторая программа выполняется дольше первой, поскольку на каждой итерации ее рабочий процесс должен отослать сообщение одному соседу и получить сообщение от другого. Это типичное противоречие между требуемым объемом памяти и временем выполнения.

Взаимодействующие равные: умножение матриц с помощью портфеля задач

В предыдущем случае управляющий процесс не участвует непосредственно в вычислениях, поэтому иногда удобно от него избавиться, позволив рабочим процессам самим распределять между собой задания или данные. Одним из общих подходов к динамическому распределению задач (данных) по процессам (потокам) является портфель задач. В этом подходе явно прослеживается единство параллелизма данных и задач. Вычислительная задача делится на конечное число подзадач, являющихся независимыми единицами работы. Каждая подзадача может выполнить однотипные действия над разными данными (яркий пример – получение элемента произведения матриц). Подзадачи нумеруются, т. е. каждому номеру соответствует свой набор данных, точнее над множеством номеров задач определяется функция, которая однозначно отражает номер задачи на соответствующий ему набор данных. Создается переменная или, если нумерация сложная, то структура для хранения номера задачи, которую следует выполнять следующей (например, номер столбца, в котором надо посчитать элементы произведения). Каждый поток независимо выполняет следующую цепочку действий: 1) обращается к портфелю задач для выяснения текущего номера задачи; 2) увеличивает портфель задач на единицу; 3) выполняет задачу, используя соответствующие данные; 4) обращается к портфелю задач для получения следующего номера задачи. Должен быть предусмотрен механизм остановки процессов при исчерпывании всего множества задач.

Таким образом, процессы (потоки) работают независимо, каждый со своей скоростью, синхронизация происходит с помощью портфеля задач, в качестве которого удобно использовать глобальную переменную.

Проблема реализации этого алгоритма в том, что доступ к портфелю задач должен быть безопасным. Оператор «получить задачу из портфеля», как правило, предполагает неделимое выполнение минимум двух операций: считывание текущего значения переменной `NextProblem` и увеличение ее на единицу. Если между этими двумя действиями работа выполняющего их потока прервется, то некоторую задачу, возможно, обработают несколько потоков.

Пример 1.6. Портфель задач

```
int NextProblem = 0; //портфель задач

thread worker (w=0; w<n) {
    Получить задачу из портфеля;
    while (задача существует) {
        Выполнить задачу;
        Получить задачу из портфеля;
    }
}
```

Умножение матриц с помощью портфеля задач

В качестве подзадачи выберем вычисление строки результирующей матрицы, тогда портфель задач – это просто счетчик строк, реализованный как глобальная переменная. Операция получения задачи из портфеля – считывание текущего значения счетчика с последующим его увеличением на единицу. Реализация должна предоставлять механизм, гарантирующий выполнение этих действий атомарно. Признаком окончания вычисления результирующей матрицы является равенство портфеля задач размерности матриц n .

Можно выбрать в качестве подзадачи вычисление одного элемента результирующей матрицы C или ее столбца.

Пример 1.7. Умножение матриц с помощью портфеля задач

```
int n= ; //объявление и инициализация n
double a[n][n], b[n][n], c[n][n];
Инициализация массивов a и b
int NextRow = 0; //портфель задач
thread worker (w=0; w<n){
    int row_current;

    //Получить задачу из портфеля, неделимо выполнив операторы
    {row_current = NextRow; NextRow++;}

    while (row_current < n) {
        //Вычисление строки матрицы C
        for (j=0; j<n; j++) {
            c[row_current][j] = 0.0;
            for (k=0; k<n; k++)
                c[row_current][j]=c[row_current][j]
                    +a[row_current][k]*b[k][j];
        }

        //Получить задачу из портфеля, неделимо выполнив операторы
        {row_current = NextRow; NextRow++;}
    }
}
```

Алгоритмы параллельного умножения матриц				
ВС с общей памятью			ВС с распределенной памятью	
Итеративный параллелизм		Взаимодействующие равные	Управляющий – рабочий	
Параллелизм по строкам (столбцам, элементам)	Параллелизм по блокам строк (столбцов, элементов): статическое распределение данных	Портфель задач: динамическое распределение данных	Управляющий – рабочий	Круговой конвейер

Приведенными примерами не исчерпываются все возможные способы распараллеливания этого простейшего алгоритма. Выбор наиболее подходящего из них тоже является отдельной весьма сложной проблемой.